

CSA0609 - DESIGN AND ANALYSIS OF ALGORITHM

Bubble Sort with Big-O Analysis

1. Aim

To write a Python program to sort an array using the **Bubble Sort algorithm** and analyze its **Big-O time and space complexity**.

2. Algorithm

1. Start with an unsorted array.
2. Compare adjacent elements.
3. If the first element is greater than the second, swap them.
4. Continue comparing adjacent elements.
5. Repeat the process for all elements.
6. After each pass, the largest unsorted element moves to its correct position.
7. Stop when the array is sorted.

3. Pseudocode

Algorithm BubbleSort(A)

Input: Array A of n elements

Output: Sorted array

1. For $i = 0$ to $n-1$ do
2. swapped $\leftarrow$ false
3. For $j = 0$ to $n-i-2$ do
4. If $A[j] > A[j+1]$ then
5. Swap $A[j]$ and $A[j+1]$
6. swapped $\leftarrow$ true
7. End If
8. End For
9. If swapped = false then

10. Break
11. End If
12. End For
13. Return A
End Algorithm

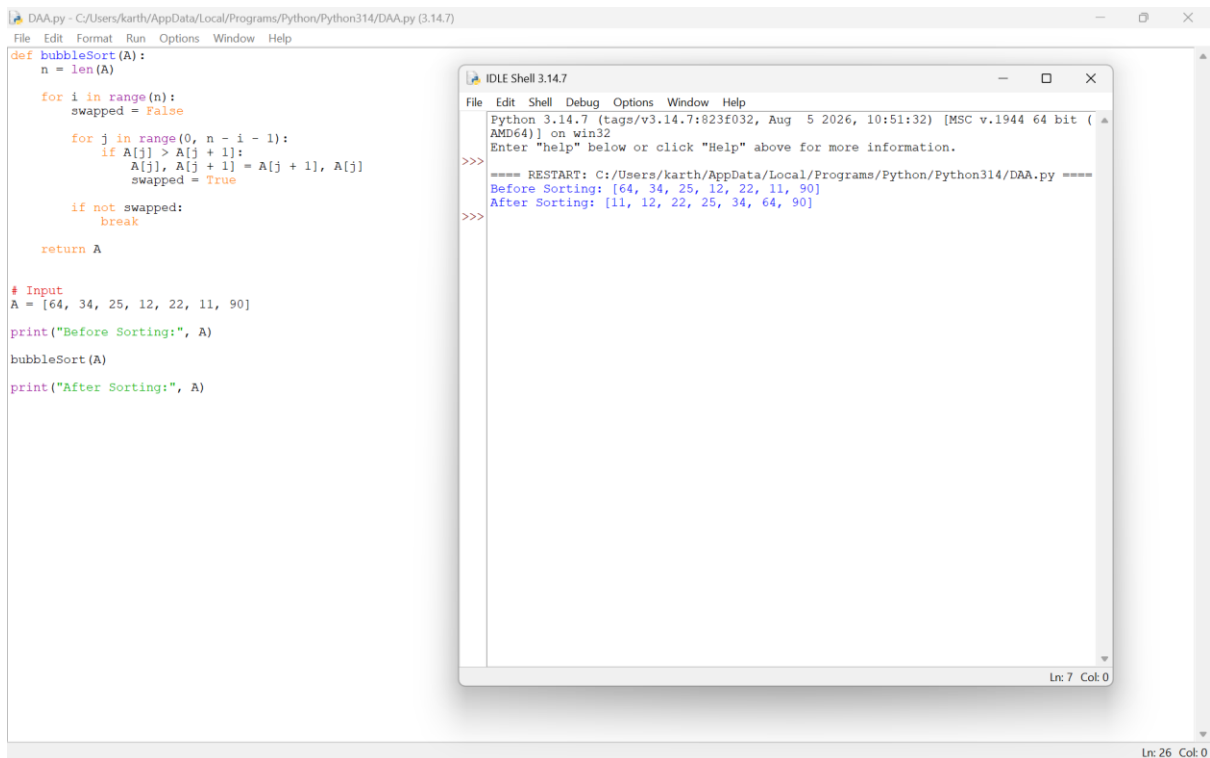
4. Python Code

```
def bubbleSort(A):  
    n = len(A)  
    for i in range(n):  
        swapped = False  
        for j in range(0, n - i - 1):  
            if A[j] > A[j + 1]:  
                A[j], A[j + 1] = A[j + 1], A[j]  
                swapped = True  
        if not swapped:  
            break  
    return A  
  
# Input  
A = [64, 34, 25, 12, 22, 11, 90]  
print("Before Sorting:", A)  
bubbleSort(A)  
print("After Sorting:", A)
```

5. Output

Before Sorting: [64, 34, 25, 12, 22, 11, 90]
After Sorting: [11, 12, 22, 25, 34, 64, 90]

6. Output Screenshot



The screenshot displays the Python IDLE environment. The main editor window shows a file named 'DAA.py' with the following code:

```
def bubbleSort(A):
    n = len(A)
    for i in range(n):
        swapped = False
        for j in range(0, n - i - 1):
            if A[j] > A[j + 1]:
                A[j], A[j + 1] = A[j + 1], A[j]
                swapped = True
        if not swapped:
            break
    return A

# Input
A = [64, 34, 25, 12, 22, 11, 90]
print("Before Sorting:", A)
bubbleSort(A)
print("After Sorting:", A)
```

The IDLE Shell window shows the execution output:

```
Python 3.14.7 (tags/v3.14.7:823f032, Aug 5 2026, 10:51:32) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
==== RESTART: C:/Users/karth/AppData/Local/Programs/Python/Python314/DAA.py ====
Before Sorting: [64, 34, 25, 12, 22, 11, 90]
After Sorting: [11, 12, 22, 25, 34, 64, 90]
```

7. Complexity Analysis

For n elements:

Case	Time Complexity
Best Case	$O(n)$
Average Case	$O(n^2)$
Worst Case	$O(n^2)$

Space Complexity: $O(1)$ because Bubble Sort is an **in-place sorting algorithm**.

7. Result

Thus, the array was successfully sorted using **Bubble Sort**. Its worst-case time complexity is $O(n^2)$ and space complexity is $O(1)$.